

THE BOURBAKI ENGINE: A GOVERNED DISCOVERY ARCHITECTURE FOR LONG-HORIZON AUTONOMOUS MATHEMATICS

MAXIMILIANO LUCIUS AND JOSÉ MINICH

ABSTRACT. Autonomous systems increasingly generate plausible research-level mathematics. A prior architecture, the BOURBAKI ENGINE, took the *admission* problem as its object: under what protocol should a generated claim be allowed to become a load-bearing dependency of future work? That question is necessary but not sufficient for *discovery*: governing accumulation says nothing about how new structure is produced over a research program that outlives any single context. This paper reorganizes the engine around a dual thesis: reliable autonomous mathematics requires two coupled but *institutionally separated* systems — a *Discovery Plane* that expands the space of mathematical possibilities (and may be exploratory, redundant, speculative, and internally contradictory) and a *Trust Plane* that governs which outputs may become reusable knowledge (and must remain conservative, auditable, and controlled by a small explicit trusted computing base). We preserve and strengthen the original contribution — claim promotion as the unit of trusted progress, edge-level soundness obligations, an epistemic separation of powers, fresh-context independence as a falsifiable hypothesis, certificate discipline (search $\neq$ evidence $\neq$ certificate $\neq$ checker), an event-sourced knowledge substrate, dependency-driven invalidation to a [SUSPECT] state, and the False Theorem Promotion Rate as the primary trust metric. Around it we specify seven modules with explicit interfaces (two stated as formal contracts), two of which are implemented and separately published as companion preprints (not yet peer-reviewed): MIRADOR, a typed, versioned representational substrate whose unit is a Mathematical Cell (prototype and case study), and PROOFCONTEXT, a dependency-aware retrieval substrate returning verifiable Evidence Bundles rather than similar text (prototype with controlled evaluation). The remaining modules — a Discovery Plane of machine-actionable operators, a formalization plane, a certified computational laboratory, an enlarged adversarial Congress, and a hierarchical research scheduler — are specified at explicitly labeled maturity levels. We replace the goal graph with a typed research hypergraph in which object type is orthogonal to epistemic state, recast the sixteen-round Bang affine plank campaign as a retrospective vertical slice (requirements elicitation, not validation of unimplemented components), and replace ambition-by-assertion with a gated readiness ladder (R0–R9) and a three-track program. The engine does not promise to solve a Millennium Prize Problem; a Millennium problem is a gated frontier stress test whose success cannot be self-declared. The reusable asset is horizontal: a verified-reasoning infrastructure applicable wherever claims are cheap to generate and expensive to trust.

1. INTRODUCTION

Mathematics has always scaled by dividing labour under a shared standard of rigor. Hilbert’s 1900 address turned open problems into public milestones [1]; Pólya systematized the heuristics of attack [2]; Lakatos showed that proofs grow by surviving refutations [3]; and the Polymath experiments demonstrated that loosely coordinated crowds can close serious theorems when the work is cut into visible, checkable pieces [19, 20]. The most successful sustained instance of industrialized rigor remains the Bourbaki collective: many mathematicians, one identity, drafts destroyed and rewritten under adversarial assembly until unanimity, and a cumulative corpus — the *Éléments* — in which every statement rests on what came before [4, 5, 6, 7, 8].

Large-language-model agents now produce competent mathematical text, search literature, write exact-arithmetic experiments, and drive proof assistants [21, 22, 23, 24]. As generation improves, two binding constraints appear, and they are different problems. The first is *admission*: deciding

under what protocol a generated argument may become a load-bearing dependency. The prior version of this architecture [59] answered exactly that, and its answer stands: the unit of progress is not a generated proof but a claim that survives a governed promotion process. The second constraint is *discovery*: producing new mathematical structure — definitions, reductions, barriers, conjectures, proofs — across a research program that spans many contexts and many months. Governing accumulation, by itself, does not generate structure. A machine that only prevents false promotions can sit forever at an empty frontier.

Principle 1 (dual thesis). Reliable autonomous mathematics requires two coupled but institutionally separated systems: a *Discovery Plane* that expands the space of mathematical possibilities, and a *Trust Plane* that governs which outputs may become reusable knowledge. The Discovery Plane may be exploratory, redundant, speculative, and internally contradictory; the Trust Plane must remain conservative, auditable, and controlled by a small explicit trusted computing base. Neither may hold the other’s authority.

This reframes the platform without weakening its original spine. The Trust Plane *is* the claim-promotion architecture, unchanged in its guarantees and still model-agnostic about how candidates are produced. What is new is that the platform now also specifies the machinery of discovery — representation, retrieval, generation operators, formalization, and certified computation — and the interfaces by which their outputs enter the promotion boundary. Two of those modules are not proposals but implemented, separately published companion substrates: the MIRADOR representational layer [60] and the PROOFCONTEXT retrieval layer [61]. The rest are specified at maturity levels we mark explicitly (§2), because the failure this paper most wants to avoid is describing unbuilt machinery in a way that reads as built.

The remaining sections state scope and contribution with maturity (§2); locate the work against related systems by capability and authority (§3); specify the two-plane architecture and its closed loop (§4); define the campaign constitution (§5), the MIRADOR substrate and the typed research hypergraph (§6), the PROOFCONTEXT substrate and Evidence Bundles (§7), the Discovery Plane operators (§8), formalization and certified computation (§9), the enlarged Congress (§10), and the hierarchical scheduler (§11); persist all of it in an event-sourced substrate with invalidation (§12); encode the Bang pilot retrospectively (§13); replace the old validation plan with a readiness ladder and three-track program (§14) and a gated Millennium stress test (§15); and state threats and limitations (§16).

2. SCIENTIFIC SCOPE AND CONTRIBUTION

The revised paper answers a broader question than its predecessor:

How can autonomous systems generate, retrieve, test, formalize, adversarially review, and cumulatively promote mathematical progress over long-horizon research programs without allowing errors to become load-bearing knowledge?

This is an architecture and research-program paper grounded in a requirements-elicitation pilot, with falsifiable evaluation protocols and explicitly separated implemented, manually exercised, specified, and aspirational components. The contributions are:

- (A) **The two-plane thesis** (§4): discovery and trust as separated authorities, with a model-agnostic promotion boundary between them.
- (B) **Edge-level soundness obligations and a typed research hypergraph** (§6): a reduction or transformation “ A follows from $B_1, \dots, B_k$ ” is itself an auditable claim, and object type (what a node *is*) is orthogonal to epistemic state (how far it has *survived*).
- (C) **Two implemented substrates with bounded contracts**: MIRADOR [60] for representation and PROOFCONTEXT [61] for retrieval, stated as interfaces rather than reproduced in full.

- (D) **A Discovery Plane of machine-actionable operators** (§8), each with preconditions, produced cell type, initial state, required retrieval context, validation obligation, and predictable failure mode.
- (E) **An epistemic separation of powers and an enlarged adversarial Congress** (§10): generation, promotion, and certification are never vested in one agent; fresh-context independence is a falsifiable hypothesis, not an intuition.
- (F) **Certificate discipline as a distinct trust layer** (§9): search $\neq$ evidence $\neq$ certificate $\neq$ checker, with a small explicit trusted computing base and an independent statement–formalization correspondence audit.
- (G) **A gated evaluation program** (§14, §15): the primary metric is the False Theorem Promotion Rate; progress toward frontier problems is gated by a readiness ladder that the engine cannot self-declare complete.

The discipline these contributions demand of *us* is a maturity ledger. Every mechanism occupies exactly the column its evidence supports; future-tense architecture is never written as if implemented.

Table 1: Maturity ledger. C = conceptual/specified; M = manually exercised (human-orchestrated pilot); P = prototype implemented; E = controlled evaluation; X = external validation. The highest supported level is marked.

Mechanism	C	M	P	E	X
MIRADOR representation [60]			•		
PROOFCONTEXT retrieval [61]				•	
Event log, hash-chaining, invalidation (in MIRADOR)			•		
Claim promotion / separation of powers		•			
Adversarial Congress (fresh-context review)		•			
Certified computation (one instance: 812,651-leaf certificate + checker)		•			
Discovery Plane operators	•				
Formalization plane (Lean/mathlib backend)	•				
Hierarchical research scheduler	•				
Campaign constitution	•				
Millennium-scale campaign	•				

No row is marked **X**: nothing here has yet survived external peer review as a system, and the Bang manuscript is not yet journal-accepted. The engine cannot promote itself to that column (§14).

3. RELATED WORK

We locate the engine not by capability alone but by *who holds which authority*. Systems that “do mathematics” can be separated by whether they generate, retrieve, formalize, certify, persist across campaigns, govern promotion, invalidate dependencies, and build theory. Table 2 records this; every 2026 entry was verified against its primary source (access date 2026-07-12; the coding table is supplementary material), and no capability coding is inferred from an abstract. In particular the certification column separates kernel-certified provers from LLM-verifier self-checking, which is not certification.

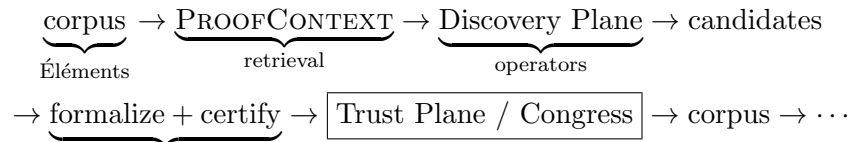
Table 2: Capability-and-authority matrix. ● = provided; ○ = partial or scoped; – = not provided. Columns: **Generate**, **Retrieve**, **Formalize**, **Certify**, **Persist** across campaigns, **Govern** promotion, **Invalidate** dependencies, build **Theory**. For the BOURBAKI ENGINE and its substrates, a cell denotes capability the architecture *specifies* (cf. the maturity ledger, Table 1); a ● appears only where a substrate implements it — retrieval via PROOFCONTEXT, invalidation via MIRADOR — and manual or conceptual capabilities are marked ○ at most, never at parity with a shipping system.

System	Gen	Retr	Form	Cert	Pers	Gov	Inv	Theory
Aletheia [25]	●	●	–	–	–	–	–	–
QED [26]	●	○	–	–	–	○	○	–
RMA [27]	●	●	–	–	○	○	–	–
Prover Agent [28]	●	–	●	●	–	–	–	–
LEAP [29]	●	–	●	●	–	–	–	–
FunSearch [23]	●	–	–	○	○	○	–	–
AlphaEvolve [31]	●	–	–	○	●	●	–	–
Lean <code>mathlib</code> [17]	–	○	●	●	●	○	○	●
iCoq [34]	–	–	●	●	●	–	●	–
<code>leanblueprint</code> [35, 36]	–	–	○	○	●	–	○	○
LeanSearch v2 [30]	○	●	–	–	–	–	–	–
TheoremGraph [38]	○	●	–	–	○	–	–	○
MaRDI [39]	–	●	–	–	●	○	–	○
Zep/Graphiti [46]	–	●	–	–	●	–	○	–
MIRADOR [60]	–	○	○	–	●	–	●	–
PROOFCONTEXT [61]	–	●	–	–	○	–	●	–
BOURBAKI ENGINE (this paper)	○	●	○	○	○	○	●	○

Three points survive this matrix. First, the recent autonomous systems [25, 26, 27] primarily optimize the probability of *finding* a correct result; none maintains a governed cross-campaign corpus. (Notably, QED enforces that its prover and verifiers share no context — an independent adoption of the separation this paper argues for.) Second, the formal-methods lineage — `mathlib` [17], iCoq [34], `leanblueprint` [35, 36] — supplies persistence, certification, and dependency tracking, but inside a single always-consistent type-checked environment that a corpus of possibly-refuted, semi-formal claims cannot assume. Third, the knowledge-graph and agent-memory systems [38, 39, 46, 47] persist and retrieve structure but hold *no* epistemic authority and do not govern promotion. The engine’s delta is the composition: it governs promotion *and* builds on a representation (MIRADOR) and retrieval (PROOFCONTEXT) that carry epistemic state and invalidate on it. Coding each entry from its primary source confirms this gap is systemic: of the systems in Table 2, none carries a live epistemic state ([OPEN]/[PROMOTED]/[REFUTED]) *together with* retrieval that hard-rejects an incompatible-but-similar match. Machine proof search — from the Logic Theory Machine [9] and AND/OR search [10] through proof planning [11] to McCune’s Robbins solution [12] — supplies the AND–OR skeleton the hypergraph generalizes; the computer-assisted lineage [13, 14, 15, 16, 18] fixes the certificate standard of §9.

4. THE TWO-PLANE ARCHITECTURE

The engine is defined by a boundary, not by its agents. Candidate mathematical objects are produced on one side and admitted, if at all, on the other. The original single boundary — “candidate → evidence → promotion → corpus” — is now embedded in a closed loop:



Everything up to the box is Discovery; the box and the corpus behind it are Trust. The membrane between them is one-way in authority: Discovery and retrieval may *propose* nodes and edges; only the Trust Plane may *promote*.

Principle 2 (governance, not generation, at the boundary). The Trust Plane does not define how claims must be generated. It defines the conditions under which claims may become load-bearing dependencies. Any generator — an end-to-end research system, a role-specialized solver, a single proof model — is a candidate source with respect to the boundary.

What changes from the predecessor is that “the engine does not define how claims are generated” is now true only *of the Trust Plane*, not of the whole platform. The integrated engine specifies the interfaces and governance of discovery — what a candidate is, where it is retrieved from, which operators produce it, how it is formalized and certified — without mandating a single generation algorithm. The two planes have opposite virtues by design: the Discovery Plane is judged by yield and diversity and is allowed to be wrong; the Trust Plane is judged by its False Theorem Promotion Rate and is allowed to be slow. Collapsing them — letting a generator certify its own output — is the precise failure the architecture exists to prevent.

5. THE CAMPAIGN CONSTITUTION

Before any graph expansion, a campaign is *constituted*: a frozen, hash-pinned document that fixes the rules of the game so that neither a generating agent nor an over-eager scheduler can quietly move them after seeing results. The constitution freezes: the exact target and admissible variants; definitions and conventions; assumptions and excluded regimes; *what counts as a solution*; allowed foundational backends and axioms; certification requirements by claim type; the novelty and attribution policy; evaluation metrics; compute and human-review budget; stop, retreat, and publication conditions; and the minimum useful deliverables.

Principle 3 (solution is defined outside the engine). For a hard target — above all a Millennium problem — “solution” cannot be defined by an internal agent verdict. The constitution must name the external process (journal acceptance, recognized community validation) that alone can close the top of the readiness ladder (§14).

The constitution is the object that makes the pilot’s costliest lesson un-repeatable by construction: a target stated as the unrestricted constant C but attacked for the one-plank-per-direction constant C^{111} is a constitution violation detectable before a single proof is written (§13).

6. MIRADOR: REPRESENTATION AND THE TYPED RESEARCH HYPERGRAPH

The substrate over which everything is stored is MIRADOR, a typed, versioned Mathematical Intermediate Representation whose unit is a *Mathematical Cell* [60]. The engine depends only on its interface, not on its internals; MIRADOR is a separately published prototype with an executable schema, a dependency-free reference implementation, twenty-eight deterministic invariant tests, and a thirteen-cell case study on the Bang problem.

Contract 6.1 (MIRADOR interface). The representation must provide: *Mathematical Cells* binding a canonical claim to its explicit context; *object type orthogonal to epistemic state*; *typed dependency and transformation edges* as first-class objects, each carrying its own soundness obligation; *versioning with semantic diffs*; claim identity as the canonical hash of a statement *and its context* (assumptions and scope participate in identity); and links between natural language, computation, and proof-assistant declarations that are recorded, never inferred from compilation.

The engine’s goal graph is generalized, in the design, to a *typed research hypergraph* over these cells. Two orthogonal axes classify every node.

Object type (what a node is). DEFINITION, QUESTION, CONJECTURE, CLAIM, REDUCTION, EQUIVALENCE, EXAMPLE, COUNTEREXAMPLE, BARRIER, EXPERIMENT, CERTIFICATE, CHECKER, HEURISTIC, TRANSLATION, PROGRAM.

Epistemic state (how far it has survived). [DRAFT], [OPEN], [EVIDENCE], [CANDIDATE], [REFUTED], [NO_GO], [FORMALIZED], [AUDITED], [PROMOTED], [SUSPECT], [RETIRED].

Principle 4 (type is not status). An object typed CLAIM and labelled “Theorem” in prose is not thereby [PROMOTED]. The literary label names the *intent*; the epistemic state names the *evidence*. Conflating them is the mechanism by which a plausible draft is cited as a theorem, and the hypergraph forbids it by keeping the axes separate.

The inverse AND–OR proof structure of the predecessor is retained as one important *subgraph*: a REDUCTION node with an AND-set of premises is exactly an AND-node, and alternative reductions to the same goal are an OR-node. But the hypergraph also represents questions, definitions, experiments, barriers, equivalences, translations, and entire research programs that the pure proof tree could not. Every REDUCTION, EQUIVALENCE, and TRANSLATION edge carries an auditable soundness obligation, checked separately from the nodes it joins — the boundary at which the pilot’s most expensive defect actually lived (§13). Two standing graph audits are specified: *acyclicity of justification* (no node reachable from its own proof obligations) and *definition–proof consistency* (the object named in a statement is byte-identical, by hash, with the object manipulated in its proof).

7. PROOFCONTEXT: DEPENDENCY-AWARE RETRIEVAL AND EVIDENCE BUNDLES

Discovery and proof both begin by asking the corpus what is already known. In mathematics, similarity is unsafe: two statements can be nearly identical in wording yet differ in a quantifier, a dimension, a symmetry assumption, a normalization, a definition version, or an epistemic status. PROOFCONTEXT is a dependency-aware retriever, layered on MIRADOR, that returns not the most similar passage but the smallest context in which a claim can be *safely used* [61]. It is a separately published prototype with a controlled evaluation.

Contract 7.1 (PROOFCONTEXT interface). Given a *Problem Signature* compiled from a natural-language query, retrieval returns a typed, hashed *Evidence Bundle*: definitions, assumptions, and exclusions; relevant claims at exact versions; dependency paths; proofs, certificates, and formal declarations; counterexamples and barriers; a provenance manifest; and epistemic states with warnings. Retrieval may propose candidate cells and edges and may rank and annotate them; it holds *no promotion authority* (the authority membrane). Filters that reject a candidate incompatible on a typed axis do so regardless of its similarity score, and both indexes and emitted bundles are invalidated version-aware.

On MATHSCOPEBENCH, an adversarial benchmark of mathematical near-misses, PROOFCONTEXT serves 0% of incompatible near-matches against 44–75% for lexical, dense, hybrid, and hybrid-without-filters retrieval, at equal or higher recall; the advantage disappears exactly when the hard filters are removed, isolating them as the cause. On downstream proxies, dependency sufficiency rises to 0.90–0.93 (from 0.40–0.43 for a flat hybrid) and correct refusal to 1.00 (from 0.00). These are prototype-scale numbers on a self-built benchmark whose near-miss categories were authored alongside the very filters they reward: the ablation isolates causal attribution but not this construct-validity risk, so a held-out or third-party benchmark is required before R1 is treated as robust. We mark them as such (Table 1); they establish retrieval readiness (R1, §14), not the engine.

8. THE DISCOVERY PLANE

The Discovery Plane is a library of machine-actionable *operators* that take retrieved Evidence Bundles and emit new MIRADOR cells in an initial epistemic state. Operators are the specified,

not-yet-implemented heart of the generative side (Table 1); the contribution here is the *contract*, which forces every operator to declare what would make its output wrong.

Table 3: Discovery operators (excerpt). Each declares admissible input, produced cell type and initial state, and the validation obligation that its output must later discharge at the Congress. All emit in [DRAFT] or [CANDIDATE]; none may emit [PROMOTED]. The full six-field per-operator contract (adding precondition and required retrieval context) is in the supplementary specification.

Operator	Produces	Validation obligation / predictable failure
SPECIALIZE	a CLAIM/EXAMPLE in a restricted regime	soundness of the restriction edge; failure: a true special case marketed as the general one.
GENERALIZE	a CONJECTURE over a wider class	a counterexample search must be scheduled; failure: over-generalization past a hidden barrier.
COUNTEREXAMPLE_FIRST	a COUNTEREXAMPLE or its absence as [EVIDENCE]	exact-arithmetic reproduction; failure: numeric near-miss mistaken for a counterexample.
NEGATE_AND_SEARCH	a BARRIER node (state [NO_GO])	the impossibility must be proved, not merely unfound; failure: absence of evidence read as evidence of absence.
DUALIZE	an EQUIVALENCE to a dual formulation	the duality edge’s soundness; failure: a lossy dual asserted as exact.
CHANGE_REPRESENTATION	a TRANSLATION cell	statement–translation correspondence audit; failure: a weaker statement encoded than intended.
TRANSFER	a CLAIM imported by analogy from another domain	the analogy is a [DRAFT] until its edge is discharged; failure: false transfer across a disanalogy.
MINIMAL_MISSING_LEMMA	a CLAIM whose proof would close a goal	acyclicity audit; failure: assuming the conclusion.
EXTREMALIZE	/ an EXPERIMENT probing extremal or perturbed cases	labelled [EVIDENCE], never proof; failure: a grid mistaken for a theorem.
STABILITY		sharpness must be proved; failure: a soft obstruction sold as a hard one.
BARRIER_SEARCH	a BARRIER capping a method family	kernel check plus correspondence audit; failure: green kernel, wrong theorem.
FORMALIZE_EARLY	a TRANSLATION to a proof assistant	triggers definition–proof re-audit of dependents; failure: definition drift.
SYNTHESIZE_DEFINITION	a DEFINITION cell (new version)	

Principle 5 (candidate generation is not novelty). An operator produces a *candidate*, not a discovery. Genuine mathematical novelty is adjudicated downstream, by a novelty-and-priority audit against the literature (§10) and by promotion. Superficial novelty by renaming is a threat the architecture must detect (§16), not a success the Discovery Plane may claim.

Two loops drive the operators. The *deductive loop* takes an [OPEN] claim, retrieves its Evidence Bundle, proposes a reduction or proof plan, generates obligations, searches for counterexamples, formalizes or proves, submits to the Congress, and receives promotion or a repair order. The *experimental loop* takes a mathematical family, generates exact instances, computes or searches, detects structure, synthesizes a conjecture or definition, attacks it with counterexamples, and extracts a certificate or a proof obligation before the same Congress. Every product of either loop enters the event log as typed events and cells.

9. FORMALIZATION AND CERTIFIED COMPUTATION

On the verification axis the engine claims no novelty: it places itself in the Flyspeck / SAT-proof / Lean-`mathlib` lineage of computer-assisted mathematics [15, 16, 14, 17] and treats that lineage’s rules as non-negotiable. What it adds is to make verification a distinct trust layer with a small explicit trusted computing base (TCB), and to audit two correspondences the lineage usually leaves implicit.

Certified computation. The admission policy requires computation to enter the corpus only in Flyspeck form: the machine-dependent step is isolated in a single finite lemma with a *static* certificate (e.g. a branch-and-bound tree whose every leaf is discharged by a sound predicate), verified by an *independent* checker (no shared code with the search, standard library only, exact rational arithmetic), with mutation tests establishing the checker’s non-vacuity and full hashes shipped with the artifact.

Principle 6 (discard the discoverer). Trust in a computational result rests on the certificate and its independent checker alone. The search, its heuristics, and its transcript may be discarded in full without affecting verifiability: search $\neq$ evidence $\neq$ certificate $\neq$ checker. Every laboratory output is labelled *heuristic*, *evidence*, *certificate*, or *proof*, and the label is load-bearing.

The pilot’s flagship instance is an 812,651-leaf exact-rational certificate for a sharp one-plank-per-direction covering constant, verified by a one-page independent checker; four hand-proved soundness lemmas and one machine-checked emptiness fact are the human/machine boundary [59]. A cautionary counterweight is on record: Ambrus’s withdrawn de-computerization of Ball’s plank theorem [55] reminds the engine that removing a certificate fails at a measurable rate, which is why certificates are never replaced by prose without a full Congress cycle.

Formalization. Formalization is specified to run as a continuous pipeline: informal statement $\rightarrow$ MIRADOR IR $\rightarrow$ formal declaration $\rightarrow$ proof state $\rightarrow$ kernel feedback $\rightarrow$ localized repair. Lean 4/`mathlib` [17] is the initial backend, but the architecture is backend-aware, not backend-identical.

Principle 7 (a kernel check is not a correspondence proof). That a formal declaration type-checks does *not* establish that it encodes the intended informal theorem. Statement–formalization correspondence is an independent claim, audited by a dedicated Congress lens (§10) and recorded in MIRADOR as `asserted`, never inferred from compilation.

10. THE CONGRESS: ROLES, POWERS, AND ADVERSARIAL LENSES

Agents are cheap; the architecture spends them on opposition. The original roles are preserved: **Stratège** (expands the graph, maintains the *plan de travail*), **Scribe** (attacks an [OPEN] node and drafts the proof), **Expérimentateur** (exact-arithmetic experiments; a numeric grid is never a proof), **Bibliothécaire** (deep literature research with adversarial source verification), **Réfuteurs** (the panel proper), **Auditeur** (standing graph-level auditor), and **Greffier** (owns certificates, checkers, and the append-only record).

Principle 8 (epistemic separation of powers). Generation, promotion, and certification are distinct authorities and are never vested in the same agent. The authority that generates a claim (i) does not promote it, (ii) does not choose its verdict, (iii) does not control the checker that verifies its computation, and (iv) does not contaminate the referee with the search transcript. This constrains *who may exercise which power*, not merely how many agents participate.

Principle 9 (unanimity at the boundary). Promotion to [PROMOTED] requires the Congress: any panelist can veto, and the veto is a *structured object* — target node/edge, concrete objection, affected dependency paths, and discharge condition — returned to the Scribe as a work order, since the Scribe holds no promotion authority over its own node.

The enlarged frontier of the two-plane engine demands new lenses beyond local correctness. The specified panel adds, as distinct fresh-context briefs not yet exercised (the pilot predates them): an *informal–formal correspondence* auditor (Principle 7); a *novelty and priority* auditor (Principle 5); a *retrieval–provenance* auditor (does the Evidence Bundle actually support what was drawn from it?); an *assumption/scope* auditor (the C versus C^{111} lens); a *checker-independence* auditor (no common-mode code between search and checker); and an *integration and invalidation* auditor (does promotion trigger the correct downstream re-audits?).

Principle 10 (fresh-context independence, as a hypothesis). Write $P(\text{defect detected} \mid c)$ for the probability a review detects a genuine defect given the reviewer’s epistemic context c . The engine’s wager is $P(\text{detected} \mid \text{fresh}) > P(\text{detected} \mid \text{shared generation history})$, along a ladder of increasing independence (self-review; watched; single fresh-context; multi-lens panel). Whether climbing this ladder measurably improves verification is an empirical question the pilot motivates ($n = 1$) but cannot settle, and that §14 proposes to measure.

11. HIERARCHICAL RESEARCH SCHEDULING

A flat frontier bandit over open nodes cannot run a years-long program. The scheduler is specified to be hierarchical, allocating a portfolio at three levels: (1) *tactical* proof obligations; (2) *research programs* lasting many cycles; and (3) *meta-program* decisions about representations and entire method families. At each level it balances exploitation, orthogonal exploration, counterexample search, formalization, certified computation, literature and priority research, [NO_GO] attempts, reproduction, and minimum publishable milestones. Within a program, the classical index

$$\text{priority}(v) = \frac{\hat{P}(\text{close } v) \cdot \text{value}(v \rightarrow R)}{\text{cost}(v)}$$

would rank tactical nodes, with $\hat{P}$ updated in bandit fashion [48, 49] and value flowing down from the root milestone R through the AND–OR substructure.

Principle 11 (the score is not evidence). A scheduler priority is a resource-allocation heuristic, never evidence of truth. It may decide what to attempt next; it may never decide what is promoted.

Three hard rules temper the index, unchanged from the predecessor: *time boxing* (an exhausted attempt returns to the Stratège with its transcript, never to the same Scribe for “one more try”); the *retreat rule* (after N failures a node is re-entered only via a new reduction edge or a [NO_GO] attempt — a proved BARRIER (a route reaching state [NO_GO]) being a first-class asset that prunes infinite search); and *milestone insurance* (the graph always contains a resourced path to a shippable *fascicule*, so a campaign degrades gracefully).

12. EVENT-SOURCED KNOWLEDGE AND INVALIDATION

A governance architecture is only as trustworthy as the record it governs. The substrate does not persist knowledge as prose.

Principle 12 (what memory holds). The engine remembers *epistemic state*, *dependency*, *provenance*, and *confidence* — where confidence is the pair (state, review level). Prose and transcripts are attachments to that record, never the record itself.

Four storage roles realize this, with a strict direction of authority; the core of the first three is what MIRADOR implements. (i) *The log is the source of truth*. An append-only sequence of campaign events (**proposed**, **evidence-added**, **promoted** recording review-independence level, **refuted**, **obligation-discharged**, **certificate-registered**, **retired**); every other store is a derived, reconstructible view [45], made tamper-evident by hash-chaining with periodic Merkle roots, the

transparent-log pattern of certificate transparency [43, 44]. Rebuildability buys time travel and auditability of the promotion process itself — the metrics of §14 are computable precisely because each promotion event carries its review level. (ii) *The graph is a truth-maintained cache.* Typed edges, each carrying its soundness obligation; its defining operation is retraction propagation: when a node is refuted or a definition it rests on is revised, invalidation propagates through the upward transitive closure, demoting every affected [PROMOTED] node to [SUSPECT] with re-audit work orders. (The propagation primitive is implemented in MIRADOR; the end-to-end promotion loop that would act on the work orders is not.) [SUSPECT] is the only state entered automatically; the only exit is back through the Congress. This is the truth-maintenance tradition [32, 33]; the closest working precedent is iCoq [34], which invalidates the upward closure of changed entities. `leanblueprint` [35, 36] propagates *good* news up a dependency graph; the engine specifies the complementary propagation of *bad* news as first-class epistemic demotion, distinct from iCoq’s recheck-invalidation, which `mathlib` can forgo only because it lives in one always-consistent environment [37]. (iii) *The corpus is immutable and content-addressed.* *Les Éléments* holds only promoted objects, frozen at promotion; identity is the canonical hash of the statement-with-context (the MIRADOR identity rule), so that “object named in the statement $\neq$ object manipulated in the proof” becomes a comparison of two hashes, and citations pin hashes in the manner of trusty URIs [41, 42]. Provenance reuses PROV-DM vocabulary [40]. (iv) *Heuristic memory has no authority.* Embeddings and traces are an index *into* the graph; retrieval must resolve to a node and its epistemic state before use. The membrane is one-way: soft memory may propose an [OPEN] node, never assert a [PROMOTED] one — the failure it prevents is a hallucinated recollection leaking into a proof. Recent agent-memory systems [46, 47] manage validity *in time* (“no longer holds”), not epistemic refutation (“proved false”), and separate no authority between soft memory and a verified corpus; for the engine that separation is a requirement.

The one binding physical requirement is transactional: a promotion commits its event, the state change, the corpus insertion, and the certificate registration atomically, or not at all. The separation of powers (Principle 8) is specified to materialize as database permissions — the generating role would hold no write access to the corpus.

13. THE BANG AFFINE PLANK VERTICAL SLICE

The pilot is a *requirements-elicitation case study*, not an evaluation, and its encoding here is *retrospective*: MIRADOR, PROOFCONTEXT, and the Discovery Plane operators were *not* implemented during the 2026 sixteen-round manual campaign on Bang’s affine plank conjecture [50, 51, 57, 56]. We show how the revised engine *would* represent that campaign’s documented events; we do not claim it ran them. With $n = 1$ the campaign motivates the mechanisms and cannot measure them.

The campaign’s outputs were a 35-page manuscript [59] (a transport-defect framework with min-max duality; a characterization of witness measures for three directions on the triangle, occupying the gap left by Gardner [52]; a rigid two-parameter family of sharp coverings; a dimension-general facet defect; and the certified covering constant of §9), a verified state-of-the-art map [57, 53, 54, 58], and a ranked program of next attacks. The design-relevant data are the *failures*, all caught before publication, none inside a proof, and each now owned by a specific mechanism and a specific typed transition.

Table 4: Retrospective encoding of pilot events — documented defects and key artifacts. Each maps to an object/edge in the typed hypergraph, an epistemic transition, and the owning mechanism.

Pilot event (defect or key artifact)	Hypergraph locus	Mechanism / transition
Object proved (C^{111}) $\neq$ object defined (C)	two DEFINITION cells, distinct context-hashes	assumption/scope lens; constitution violation (§5)

Pilot event	Hypergraph locus	Mechanism
Invalid reduction edge (proved for C^{11} , stated for C)	REDUCTION edge obligation	edge-level audit; $\rightarrow$ [REFUTED] edge (§6)
Circular termination argument	CLAIM reachable from its own obligations	acyclicity audit; static-certificate template (§9)
Priority claim (“first non-concurrent triple”) falsified	CLAIM vs COUNTEREXAMPLE	novelty/priority lens (§10)
Six internal audits missed what one external pass caught	review-level metadata on the promoted events	fresh-context panels; <i>la retraite</i> (Principle 10)
Sign-cell typo in a soundness lemma; degenerate-direction gap	CLAIM/CHECKER with a  flag	multi-lens re-derivation; dangerous-bend discharge
812,651-leaf certificate + independent checker	CERTIFICATE, CHECKER cells	certificate discipline (Principle 6)
Route obstructions (Bang sign argument; symplectic ceiling)	BARRIER nodes (state [NO_GO])	retreat rule; barrier as asset (§11)

Conversely, what never failed — ~ 22 adversarially re-derived computations, massive exact-arithmetic cross-checks, the certificate/checker pair — is exactly what had already been subjected to opposition and exact arithmetic. The working hypothesis — *errors concentrate where claims meet language, so that is where adversaries should be spent* — is consistent with the one campaign on record but remains a hypothesis until the node-versus-edge error share of §14 is measured across many.

14. VALIDATION LADDER AND THREE-TRACK PROGRAM

We replace any impression that the next step is “solve a Millennium problem” with a gated readiness ladder. Each rung is a falsifiable claim about the system, and the engine cannot self-declare the top rungs.

Table 5: The readiness ladder. Current status is marked against Table 1:
R0–R1 have prototype evidence; R2–R9 are open.

Rung	Readiness criterion
R0	<i>Representation</i> : MIRADOR schemas, identity, and versioning operate on real claims. (prototype)
R1	<i>Retrieval</i> : PROOFCONTEXT rejects incompatible near-matches and reconstructs usable Evidence Bundles. (prototype eval)
R2	<i>Governance</i> : events, permissions, promotion, veto, invalidation, and deterministic replay work end to end.
R3	<i>Discovery</i> : the system regenerates nontrivial known results under contamination controls and records failed routes.
R4	<i>Open-problem</i> : a genuinely new lemma, counterexample, reduction, or barrier on a bounded open problem.
R5	<i>External</i> : at least one result survives independent expert or journal review.
R6	<i>Millennium mapping</i> : a target-specific constitution and a verified state-of-the-art atlas are released.
R7	<i>Millennium frontier</i> : a novel, externally reviewed result advances a recognized subproblem or proves a useful barrier.
R8	<i>Candidate resolution</i> : a complete candidate proof/disproof with closed dependencies and formal/certified artifacts, externally reviewed.
R9	<i>Community validation</i> : acceptance through the external mathematical process. <i>The engine cannot promote itself to this state.</i>

Three concurrent tracks feed the ladder. **Track A (engineering and reconstruction)** uses the Bang campaign as the end-to-end integration laboratory. **Track B (blind scientific evaluation)** uses problems supplied by external mathematicians — withheld solutions or unpublished subproblems — under contamination and information-access controls, which is the only honest way

to measure discovery. **Track C (Millennium preparation)** is initially restricted to constitution, literature atlas, formalized definitions, barriers, equivalences, certified computational frontiers, and ranked subproblems; it may advance to direct frontier research only after predetermined gates from Tracks A and B are passed.

Metrics by plane. The primary trust metric remains the *False Theorem Promotion Rate* (FTPR): the fraction of [PROMOTED] promotions that are, in fact, wrong. It is never collapsed with discovery metrics into a single score. Because R2–R9 are open, no promotions exist yet and the FTPR is currently *undefined*; its estimator is nonetheless specified — seeded known-false claims, Track-B blind ground truth, and post-hoc external review supply the numerator over all [PROMOTED] events — so it becomes measurable exactly when governance (R2) runs. Around it: (*Discovery*) useful-candidate rate, time-to-counterexample, novel-result yield, program diversity, reduction depth, theory compression, barrier discovery, cross-campaign reuse; (*Retrieval/representation*) assumption-compatible retrieval precision, Evidence-Bundle completeness, definition/version accuracy, dependency-path recall, stale-claim serving rate, informal–formal alignment accuracy; (*Trust*) FTPR, defect detection and escape rates, false-rejection rate, node-versus-edge error share, independent reproduction rate, formalization coverage, re-audit latency, and cost per correctly promoted claim. A system that solves more problems while raising its FTPR is, by this architecture’s standard, worse.

15. THE MILLENNIUM-SCALE STRESS TEST

A Millennium Prize Problem is the intended stress test at the top of the ladder, not a promised deliverable, and the title of this paper makes no such promise. Its role is to supply adversarial pressure no synthetic benchmark can, at a target where “solution” is defined entirely outside the engine (Principle 3). Consistent with milestone insurance (§11), a Millennium campaign is structured so its floor ships regardless of the frontier: a verified state-of-the-art atlas and formalized definitions (R6) are a survey *fascicule* that is valuable even if no new theorem is proved; a frontier result (R7) — and a proved BARRIER (state [NO.GO]) is valued at parity with a positive result — is the aspirational ceiling; and R8–R9 depend on an external process the engine cannot short-circuit. The ceiling is upside, never the plan.

16. THREATS TO VALIDITY AND LIMITATIONS

The engine does not manufacture insight, novelty, tractability, or a Millennium resolution; a campaign is bounded by the best reduction any Stratège proposes, and by representation bottlenecks, formalization cost, retrieval error, model homogenization, benchmark contamination, human-expert bottlenecks, and compute limits. Against known failure modes, we classify the architecture’s response as **prevent**, **detect**, **record**, **mitigate**, or **open**.

Table 6: Threat model.

Threat	Response	Mechanism
Hallucinated citations	detect	Bibliothécaire source verification; retrieval-provenance lens
False novelty / renaming	detect	novelty-and-priority lens (Principle 5)
Statement/definition/proof mismatch	prevent	context-hash identity; definition–proof audit
Invalid transformation/reduction edge	detect	edge-level soundness obligation
Formalizing a weaker statement	detect	correspondence audit (Principle 7)
Assumption/scope leakage	detect	assumption/scope lens; hard retrieval filters
Stale retrieval	prevent	version-aware invalidation of indexes and bundles
Corrupted graph edges	record	append-only log; deterministic replay

Threat	Response	Mechanism
Retrieval poisoning / prompt injection	mitigate	authority membrane; no promotion from soft memory
Checker/searcher common-mode bug	detect	checker-independence lens; mutation tests
Dependency cycles	prevent	acyclicity-of-justification audit
Agent collusion / shared-context bias	mitigate	separation of powers; fresh-context panels
Premature convergence to one program	mitigate	hierarchical portfolio; orthogonal-exploration budget
Benchmark contamination	mitigate	Track B blind problems; contamination controls
Compute-driven publication bias	record	every promotion event logs its review level and cost
Governance capture	open	the top of the ladder is external by construction

The central limitation is evidential: two modules are implemented prototypes, one computation is certified, the governance and Congress are so far only manually exercised, and the discovery, formalization, and scheduling planes are specified, not built (Table 1). The immediate work is to climb R2–R4 under the three-track program and to test the fresh-context hypothesis (Principle 10) by varying reviewer independence.

17. CONCLUSION

The transition this paper makes is deliberate: a claim-promotion architecture, preserved intact as the Trust Plane, becomes a governed *discovery* architecture supported by an implemented representation (MIRADOR) and retrieval (PROOFCONTEXT) substrate, and evaluated through readiness gates rather than assertion. The final position is meant to be ambitious but falsifiable.

Principle 13 (position). The BOURBAKI ENGINE does not claim to manufacture a Millennium solution. It specifies and evaluates the institutional, representational, computational, and verification machinery required for autonomous mathematical research to accumulate frontier progress *without epistemic collapse* — the state in which cheap generation silently fills a corpus with load-bearing falsehoods.

Its reusable asset is horizontal: governed goal graphs, adversarial promotion boundaries, certificate discipline, and dependency-driven invalidation are a verified-reasoning infrastructure applicable wherever claims are cheap to generate and expensive to trust. Each campaign is intended to calibrate the scheduler’s priors and harden the audit checklists, so that — if the effect holds — the engine that finishes a milestone is better calibrated than the one that started it.

APPENDIX A. INTERNAL LANGUAGE (NORMATIVE GLOSSARY)

Bourbaki’s working culture survives in its documents, and the engine adopts its vocabulary because names carry norms. The glossary is normative for platform documents. *Les Éléments*: the corpus of [PROMOTED] nodes with their proofs and certificates, the only citable ground truth. *Fascicule*: a shippable closed milestone, the publication quantum. *La Tribu*: the append-only campaign log, the single source of truth of which graph, corpus, and indexes are derived views. *Le Congrès*: the refutation panel of §10. *Le veto*: a structured block on promotion; unanimity is required at the [PROMOTED] boundary. *Tournant dangereux* (

extending, institutionalizing fresh eyes. *Le cobaye*: a deliberately naive agent given a draft cold, to test self-containedness before the Congress. *Abus de langage*: sanctioned informal shorthand, legal only when hyperlinked to the rigorous statement it abbreviates. *Plan de travail*: the Stratège’s prioritized frontier snapshot, the only to-do list.

APPENDIX B. OBJECT TYPES AND EPISTEMIC STATES

Object type answers “what is this node?” and is fixed at creation: DEFINITION, QUESTION, CONJECTURE, CLAIM, REDUCTION, EQUIVALENCE, EXAMPLE, COUNTEREXAMPLE, BARRIER, EXPERIMENT, CERTIFICATE, CHECKER, HEURISTIC, TRANSLATION, PROGRAM. Epistemic state answers “how far has it survived?” and changes only through logged events: [DRAFT] $\rightarrow$ [OPEN] $\rightarrow$ [EVIDENCE]/[CANDIDATE] $\rightarrow$ [FORMALIZED] $\rightarrow$ [AUDITED] $\rightarrow$ [PROMOTED], with [REFUTED] and [NO_GO] as terminal-for-a-route outcomes, [SUSPECT] as the only automatically entered state (via invalidation), and [RETIRED] for superseded versions (assigned as part of the promoting event that mints the new version, not by automatic demotion). The two axes are independent: a CONJECTURE may be [PROMOTED] (as a conjecture, with that status), and a CLAIM labelled “Theorem” may sit at [DRAFT] indefinitely (Principle 4).

REFERENCES

- [1] D. Hilbert, *Mathematische Probleme*, Nachr. Ges. Wiss. Göttingen, Math.-Phys. Kl. (1900), 253–297.
- [2] G. Pólya, *How to Solve It*, Princeton University Press, 1945.
- [3] I. Lakatos, *Proofs and Refutations*, Cambridge University Press, 1976.
- [4] N. Bourbaki, *Éléments de mathématique*, Hermann (later Masson, Springer), Paris, 1939–.
- [5] M. Mashaal, *Bourbaki: A Secret Society of Mathematicians*, American Mathematical Society, 2006.
- [6] L. Corry, *Nicolas Bourbaki and the concept of mathematical structure*, Synthese **92** (1992), 315–348.
- [7] A. Weil, *The Apprenticeship of a Mathematician*, Birkhäuser, 1992.
- [8] L. Beaulieu, *A Parisian café and ten proto-Bourbaki meetings (1934–1935)*, Math. Intelligencer **15** (1993), no. 1, 27–35.
- [9] A. Newell, J. C. Shaw, H. A. Simon, *Empirical explorations of the Logic Theory Machine*, Proc. Western Joint Computer Conference (1957), 218–230.
- [10] N. J. Nilsson, *Problem-Solving Methods in Artificial Intelligence*, McGraw-Hill, 1971.
- [11] A. Bundy, *The use of explicit plans to guide inductive proofs*, Proc. CADE-9, LNCS **310**, Springer (1988), 111–120.
- [12] W. McCune, *Solution of the Robbins problem*, J. Automated Reasoning **19** (1997), 263–276.
- [13] K. Appel, W. Haken, *Every planar map is four colorable. Part I: Discharging*, Illinois J. Math. **21** (1977), 429–490.
- [14] G. Gonthier, *Formal proof—the four-color theorem*, Notices Amer. Math. Soc. **55** (2008), 1382–1393.
- [15] T. Hales et al., *A formal proof of the Kepler conjecture*, Forum of Mathematics, Pi **5** (2017), e2.
- [16] M. J. H. Heule, O. Kullmann, V. W. Marek, *Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer*, Proc. SAT 2016, LNCS **9710**, Springer (2016), 228–245.
- [17] The mathlib Community, *The Lean mathematical library*, Proc. 9th ACM SIGPLAN Conf. Certified Programs and Proofs (CPP 2020), 367–381.
- [18] N. G. de Bruijn, *The mathematical language AUTOMATH*, Symposium on Automatic Demonstration, LNM **125**, Springer (1970), 29–61.
- [19] T. Gowers, M. Nielsen, *Massively collaborative mathematics*, Nature **461** (2009), 879–881.
- [20] D. H. J. Polymath, *A new proof of the density Hales–Jewett theorem*, Ann. of Math. **175** (2012), 1283–1327.
- [21] S. Yao et al., *Tree of Thoughts: deliberate problem solving with large language models*, NeurIPS 36 (2023).
- [22] T. H. Trinh, Y. Wu, Q. V. Le, H. He, T. Luong, *Solving olympiad geometry without human demonstrations*, Nature **625** (2024), 476–482.
- [23] B. Romera-Paredes et al., *Mathematical discoveries from program search with large language models*, Nature **625** (2024), 468–475.

- [24] A. Davies et al., *Advancing mathematics by guiding human intuition with AI*, Nature **600** (2021), 70–74.
- [25] T. Feng, T. H. Trinh, G. Bingham, et al. (Google DeepMind), *Towards Autonomous Mathematics Research* (the Aletheia system), arXiv:2602.10177 (2026).
- [26] C. An, Q. Ye, M. Pan, J. Zhang, *QED: an open-source multi-agent system for generating mathematical proofs on open problems*, arXiv:2604.24021 (2026).
- [27] Z. Zhao, B. Yuan, J. Choi, Y. Chen, *RMA: an agentic system for research-level mathematical problems*, arXiv:2605.22875 (2026).
- [28] Baba, Liu, Kurita, Sannai, *Prover Agent: an agent-based framework for formal mathematical proofs*, arXiv:2506.19923 (2025).
- [29] Kung, Song, Hwang, et al., *LEAP: supercharging LLMs for formal mathematics with agentic frameworks*, arXiv:2606.03303 (2026).
- [30] Gao, Sun, Jiang, et al., *LeanSearch v2: global premise retrieval for Lean 4 theorem proving*, arXiv:2605.13137 (2026).
- [31] Novikov, Balog, et al. (Google DeepMind), *AlphaEvolve: a coding agent for scientific and algorithmic discovery*, Google DeepMind technical report (2025), deepmind.google/discover/blog/alphaevolve.
- [32] J. Doyle, *A truth maintenance system*, Artificial Intelligence **12** (1979), no. 3, 231–272.
- [33] J. de Kleer, *An assumption-based TMS*, Artificial Intelligence **28** (1986), 127–162.
- [34] A. Celik, K. Palmiskog, M. Gligoric, *iCoq: regression proof selection for large-scale verification projects*, Proc. ASE 2017.
- [35] P. Massot, *leanblueprint: a plasTeX plugin for writing formalization blueprints for Lean 4 projects*, software.
- [36] T. Tao, *Formalizing the proof of PFR in Lean4 using Blueprint: a short tour*, blog post, November 2023.
- [37] F. van Doorn, G. Ebner, R. Y. Lewis, *Maintaining a library of formal mathematics*, Proc. CISM 2020, LNCS **12236**, Springer (2020); arXiv:2004.03673.
- [38] S. Kurgan, E. Wang, E. Leonen, et al., *TheoremGraph: bridging formal and informal mathematics*, arXiv:2606.25363 (2026).
- [39] M. Schubotz, E. Ferrer, J. Stegmüller, et al., *Bravo MaRDI: a Wikibase powered knowledge graph on mathematics*, arXiv:2309.11484 (2023).
- [40] L. Moreau, P. Missier (eds.), *PROV-DM: the PROV data model*, W3C Recommendation, 30 April 2013.
- [41] T. Kuhn, M. Dumontier, *Trusty URIs: verifiable, immutable, and permanent digital artifacts for linked data*, Proc. ESWC 2014, LNCS **8465**, Springer (2014), 395–410.
- [42] T. Kuhn et al., *Decentralized provenance-aware publishing with nanopublications*, PeerJ Computer Science **2** (2016), e78.
- [43] B. Laurie, A. Langley, E. Kasper, *Certificate transparency*, RFC 6962, IETF, 2013.
- [44] R. Cox, *Transparent logs for skeptical clients*, research.swtch.com/tlog, 2019.
- [45] M. Fowler, *Event sourcing*, martinfowler.com, 2005.
- [46] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, D. Chalef, *Zep: a temporal knowledge graph architecture for agent memory*, arXiv:2501.13956 (2025).
- [47] N. Yadav, *Temporal validity in retrieval memory*, arXiv:2606.26511 (2026).
- [48] P. Auer, N. Cesa-Bianchi, P. Fischer, *Finite-time analysis of the multiarmed bandit problem*, Machine Learning **47** (2002), 235–256.
- [49] C. Browne et al., *A survey of Monte Carlo tree search methods*, IEEE Trans. Comput. Intell. AI in Games **4** (2012), 1–43.
- [50] T. Bang, *A solution of the “plank problem”*, Proc. Amer. Math. Soc. **2** (1951), 990–993.
- [51] K. Ball, *The plank problem for symmetric bodies*, Invent. Math. **104** (1991), 535–543.
- [52] R. J. Gardner, *Relative width measures and the plank problem*, Pacific J. Math. **135** (1988), 299–312.
- [53] A. Akopyan, R. Karasev, F. Petrov, *Bang’s problem and symplectic invariants*, J. Symplectic Geom. **17** (2019), no. 6, 1579–1611.
- [54] A. Glazyrin, R. Karasev, A. Polyanskii, *Covering by planks and avoiding zeros of polynomials*, Int. Math. Res. Not. IMRN (2023); arXiv:2112.05382.
- [55] G. Ambrus, *A short proof of the plank theorem for symmetric bodies*, arXiv:2203.11260 (2022); withdrawn by the author.
- [56] W. Verreault, *Plank theorems and their applications: a survey*, Bull. London Math. Soc. (2026); arXiv:2203.05540.

- [57] A. Bakaev, A. Yehudayoff, *A lower bound for the total relative width of planks covering a convex body*, arXiv:2602.20290 (2026).
- [58] A. D. Martínez, O. Ortega-Moreno, *A solution to the strong polarization problem*, arXiv:2605.28744 (2026).
- [59] M. Lucius, *Transport and tiling bounds for the affine plank problem on the triangle*, manuscript (2026), 35 pp.; repository `maximilianolucius/bang-plank-affine-triangle`.
- [60] M. Lucius, *MIRADOR: a typed, versioned intermediate representation for auditable mathematical knowledge and discovery*, software and preprint (2026), Zenodo `10.5281/zenodo.21324524`; repository `maximilianolucius/mirador`.
- [61] M. Lucius, *ProofContext: dependency-aware retrieval of verifiable context for mathematical reasoning*, software and preprint (2026), Zenodo `10.5281/zenodo.21325288`; repository `maximilianolucius/proofcontext`.

INSTITUTO TECNOLÓGICO DE BUENOS AIRES, CABA, ARGENTINA

Email address: `maximiliano.lucius@tutamail.com`

INSTITUTO TECNOLÓGICO DE BUENOS AIRES, CÓRDOBA, ARGENTINA

Email address: `jminich@gmail.com`